❶ The name of the container

❷ Defines the username for the admin user

❸ Defines the password for the admin user

❹ Defines the name of the database to be created

❺ Exposes the database to port 5432 on your machine

❻ The PostgreSQL container image pulled by Docker Hub

Compared to how you ran the Catalog Service container, you'll notice a few new elements. First, the Docker image from which you run a container (`postgres:14.4`) is not created by you—it's pulled from the Docker Hub container registry (configured by default when you install Docker).

The second new thing is passing environment variables as arguments to the container. PostgreSQL accepts a few environment variables that are used during the container's creation to configure a database.

NOTE In this book, I won't cover how to configure storage in Docker (*volumes*). This means that all the data saved in your local PostgreSQL container will be lost once you remove the container. It might seem counterintuitive given the topic of this chapter, but any storage-related concern will be handled by the cloud provider in production, so you won't have to deal with it yourself. If you need to add persistent storage to your local containers, though, you can read how to use volumes in the official Docker documentation (https://docs.docker.com).

In the next section, you'll see how to add data persistence to a Spring Boot application using Spring Data JDBC and PostgreSQL.

NOTE If you need to, you can stop the container with `docker stop polar-postgres` and start it again with `docker start polar-postgres`. If you want to start over, you can remove the container with `docker rm -fv polar-postgres` and create it again with the previous `docker run` command.

5.2 Data persistence with Spring Data JDBC